

Compilers' Theory

Lecture 1

Prof. Zaki Taha

Course Coordinator
Dr. Mohamed Ali
mohamed.abdelghafour@cis.asu.edu.eg

Special Thanks

Credit to Dr. Sally Saad for the content

Lecture Rules

- Please set mobile phones to silent mode.
- Please arrive on Time.

Suggested Book

- Introduction to Compilers and Language Design, 2nd edition
- https://dthain.github.io/books/compiler/



Grades

- 50 Final
- 15 Midterm
- 20 Project
- 10 Year Works
- 5 Quiz

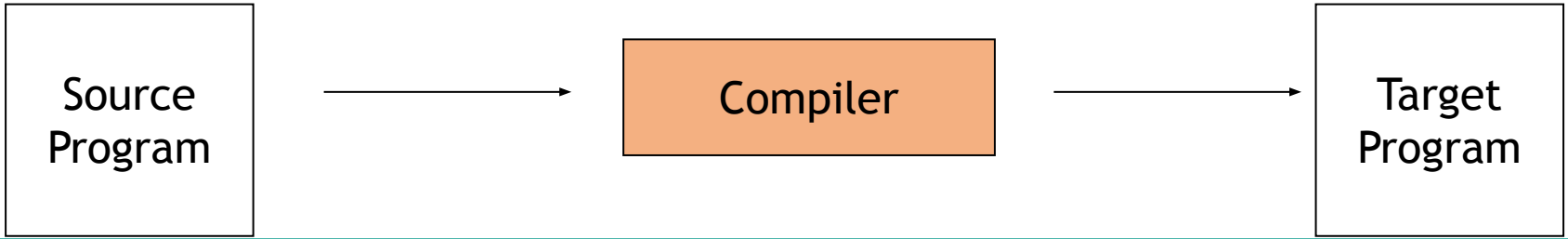




Introduction

What is a Compiler?

- A compiler **translates** a program in a **source language** to a program in a **target language**.
- The most well known form of a compiler is one that translates a high-level language like C into the native assembly language of a machine so that it can be executed.
- And of course there are compilers for other languages like C++, Java and C#, and many others.



What is a Compiler?



- Error Prevention: Assists programmers by catching and flagging errors at compile time to avoid fatal runtime failures.
- The Strictness Trade-off: Stricter languages increase compile-time errors, making development harder but ensuring program correctness.
- Real-World Example (Ada): Infamously difficult to compile without errors, but highly trusted to run safety-critical systems like the Boeing 777 aircraft.

Compiler Vs. Interpreter



- Compiler: Translates source code into a target language (like machine code) before execution.
- Interpreter: Reads and executes a program directly on the fly, without emitting a translation.
    - Languages like Python and Ruby are typically executed by an interpreter that reads the source code directly.

Why should you study compilers?



- Become a Better Programmer.
- Build Specialized Tools.
- Create Custom Languages.
- Contribute to Compiler Evolution.
- Solve Fun Challenges.

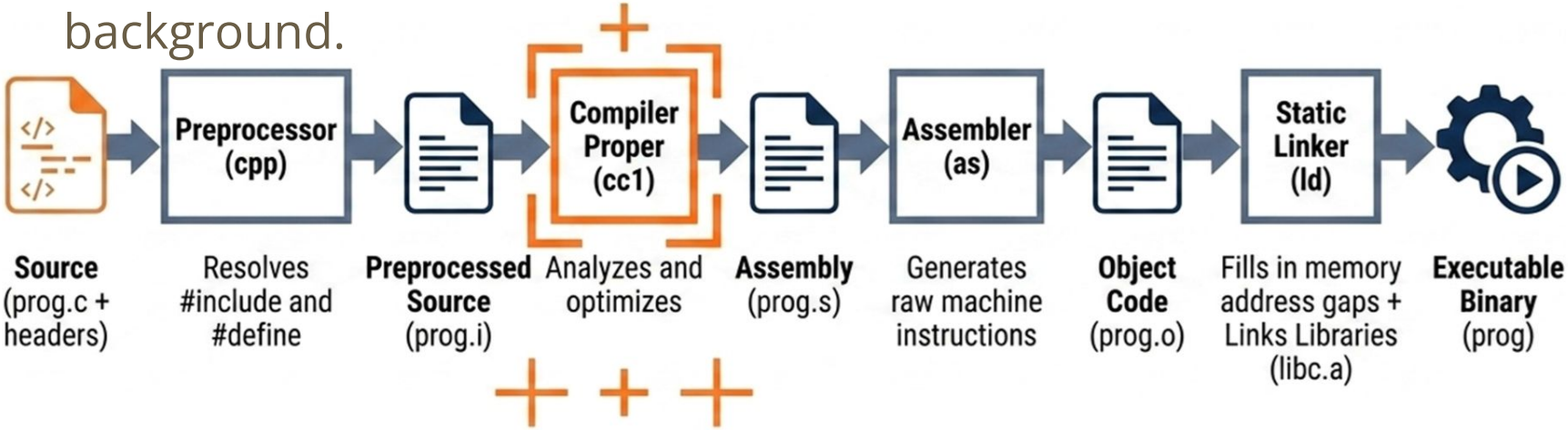
What's the best way to learn about compilers?



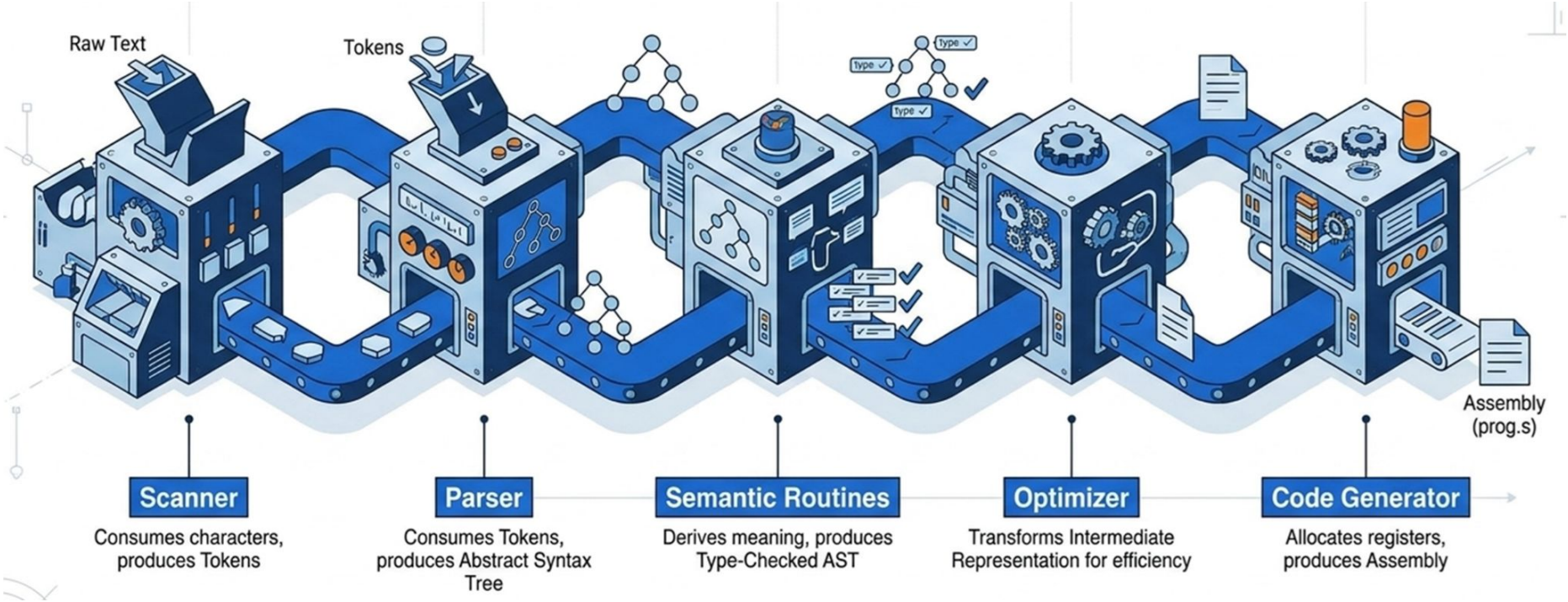
- The best way to learn about compilers is to **write your own compiler** from beginning to end.

The Compiler Toolchain

- A compiler is one component in a toolchain of programs used to create executables from source code.
- Typically, when you invoke a single command to compile a program, a whole sequence of programs are invoked in the background.



Stages Within a Compiler Proper



Stages Within a Compiler Proper



- **Scanner:** Reads character streams and groups them into structured tokens.
- **Parser:** Checks tokens against a grammar to build an Abstract Syntax Tree (AST).
- **Semantic Routines:** Verify program logic (type checking) and convert the AST to an Intermediate Representation (IR).
- **Optimizer:** Refines the IR to make code smaller, faster, and more resource-efficient.
- **Code Generator:** Translates optimized IR to concrete assembly, managing instruction selection and register allocation.

Example Compilation



- Suppose we wish to compile this fragment of code into assembly:
  - height = (width+56) * factor(foo);

Example Compilation: Scanner

- The first stage of the compiler (the scanner) will read in the text of the source code character by character, identify the boundaries between symbols, and emit a series of tokens. Each token is a small data structure that describes the nature and contents of each symbol:
  - Lexeme: The actual word or text written in the code.
  - Token: The grammatical category or label assigned to that text.



Example Compilation: Scanner

- At this stage, the purpose of each token is not yet clear.
- For example:
  - factor and foo are simply known to be identifiers, even though one is the name of a function, and the other is the name of a variable.
  - Likewise, we do not yet know the type of width, so the + could potentially represent integer addition, floating point addition, string concatenation, or something else entirely.



Example Compilation: Parser



- The next step is to determine whether this sequence of tokens forms a valid program.
- The parser does this by looking for patterns that match the grammar of a language.
- Suppose that our compiler understands a language with the following grammar:

1. EXPR -> EXPR **+** EXPR
2. EXPR -> EXPR ***** EXPR
3. EXPR -> EXPR **=** EXPR
4. EXPR -> **id (** EXPR **)**
5. EXPR -> **(** EXPR **)**
6. EXPR -> **id**
7. EXPR -> **int**

Example Compilation: Parser



- Each line of the grammar is called a rule, and explains how various parts of the language are constructed.
- Rules 1-3 indicate that an expression can be formed by joining two expressions with operators.
- Rule 4 describes a function call.
- Rule 5 describes the use of parentheses.
- Finally, rules 6 and 7 indicate that identifiers and integers are atomic expressions.

1. EXPR -> EXPR **+** EXPR
2. EXPR -> EXPR ***** EXPR
3. EXPR -> EXPR **=** EXPR
4. EXPR -> **id (** EXPR **)**
5. EXPR -> **(** EXPR **)**
6. EXPR -> **id**
7. EXPR -> **int**

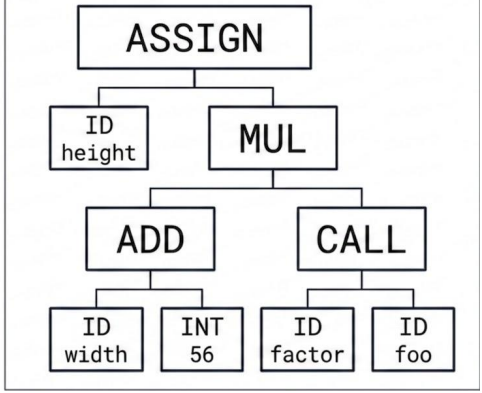
Example Compilation: Parser



- The parser looks for sequences of tokens that can be replaced by the left side of a rule in our grammar.
- Each time a rule is applied, the parser creates a node in a tree, and connects the sub-expressions into the abstract syntax tree (AST).
- The AST shows the structural relationships between each symbol: addition is performed on width and 56, while a function call is applied to factor and foo.

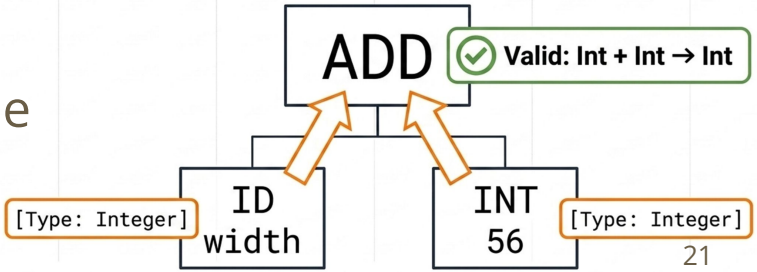
Grammar G1

```
1. expr -> expr + expr
2. expr -> expr * expr
3. expr -> expr = expr
4. expr -> id ( expr )
5. expr -> ( expr )
6. expr -> id
7. expr -> int
```



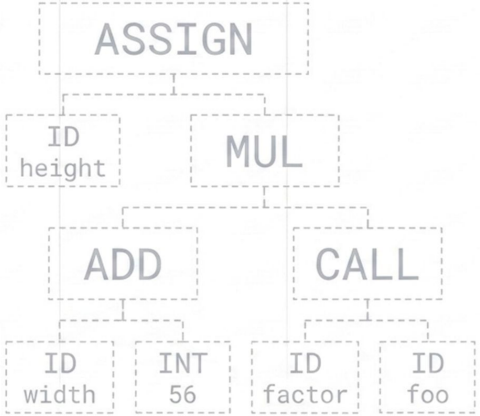
Example Compilation: Semantic Routines

- With this data structure in place, we are now prepared to analyze the meaning of the program.
- The semantic routines traverse the AST and derive additional meaning by relating parts of the program to each other and to the definition of the programming language.
- An important component of this process is type checking, in which the type of each expression is determined, and checked for consistency with the rest of the program.
- To keep things simple here, we will assume that all of our variables are plain integers.



Example Compilation: Semantic Routines

- To generate linear intermediate code, we perform a post-order traversal of the AST and generate an IR instruction for each node in the tree.
- A typical IR looks like an abstract assembly language, with load/store instructions, arithmetic operations, and an infinite number of registers.
- For example, this is a possible IR representation of our example program



LOAD	$56 -> r1
LOAD	width -> r2
IADD	r1, r2 -> r3
ARG	foo
CALL	factor -> r4
IMUL	r3, r4 -> r5
STOR	r5 -> height

Example Compilation: Optimizer



- The intermediate representation is where most forms of optimization occur.
- Dead code is removed, common operations are combined, and code is generally simplified to consume fewer resources and run more quickly.

Example Compilation: Code Generator

- Finally, the intermediate code must be converted to the desired assembly code.

Linear IR		X86-64 Assembly
LOAD width -> r2	———	MOVQ width, %rax
IADD $56, r2 -> r3	———	ADDQ $56, %rax
(Save to temporary)	———	MOVQ %rax, -8(%rbp)
ARG foo	———	MOVQ foo, %edi
CALL factor -> r4	———	CALL factor
(Retrieve temporary)	———	MOVQ -8(%rbp), %rbx
IMUL r3, r4 -> r5	———	IMULQ %rbx
STOR r5 -> height	———	MOVQ %rax, height

Scanning

What is Scanning? (Lexical Analysis)



- Scanning is the first phase of a compiler proper, acting as the interface between the text file and the syntax analyzer (parser).
- The Analogous Concept:
  - **Natural Language:** Grouping individual letters into meaningful words.
  - **Compiler:** Grouping individual characters into syntactic "words" called Tokens.
- Input vs. Output:
  - **Input:** A continuous, low-level stream of characters (ASCII/Unicode).
  - **Output:** A high-level stream of discrete token structures.

The Function of a Scanner



- Reading characters from the source code (lexemes) and form them into logical units called **tokens.**
- Tokens are logical entities defined as an enumerated type.

Kinds of Tokens



- Scanning is the process of identifying tokens from the raw text source code of a program.
- Most programming languages partition their vocabulary into five distinct token categories:
  a. Keywords.
  b. Identifiers.
  c. Numbers.
  d. Strings.
  e. Comments & Whitespace.

Kinds of Tokens



- Keywords:
  - Reserved words forming the language's syntax structural backbone (e.g., while, if, class, return).
- Identifiers:
  - User-defined names for variables, functions, and classes (e.g., width, height, factor).
- Numbers:
  - Numeric constants formatted as integers, floats, or hexadecimals (e.g., 123, 3.14, 0x1A).
- Strings:
  - Literal character sequences wrapped in delimiters (e.g., "hello world\n")
- Comments & Whitespace:
  - Formatting blocks ignored by the parser but critical for human readability

Scanning a Source File Example

w	h	i	l	e		(	1	3	7		<		i	)	\n	\t	+	+	i	;

Scanning a Source File Example



Scanning a Source File Example



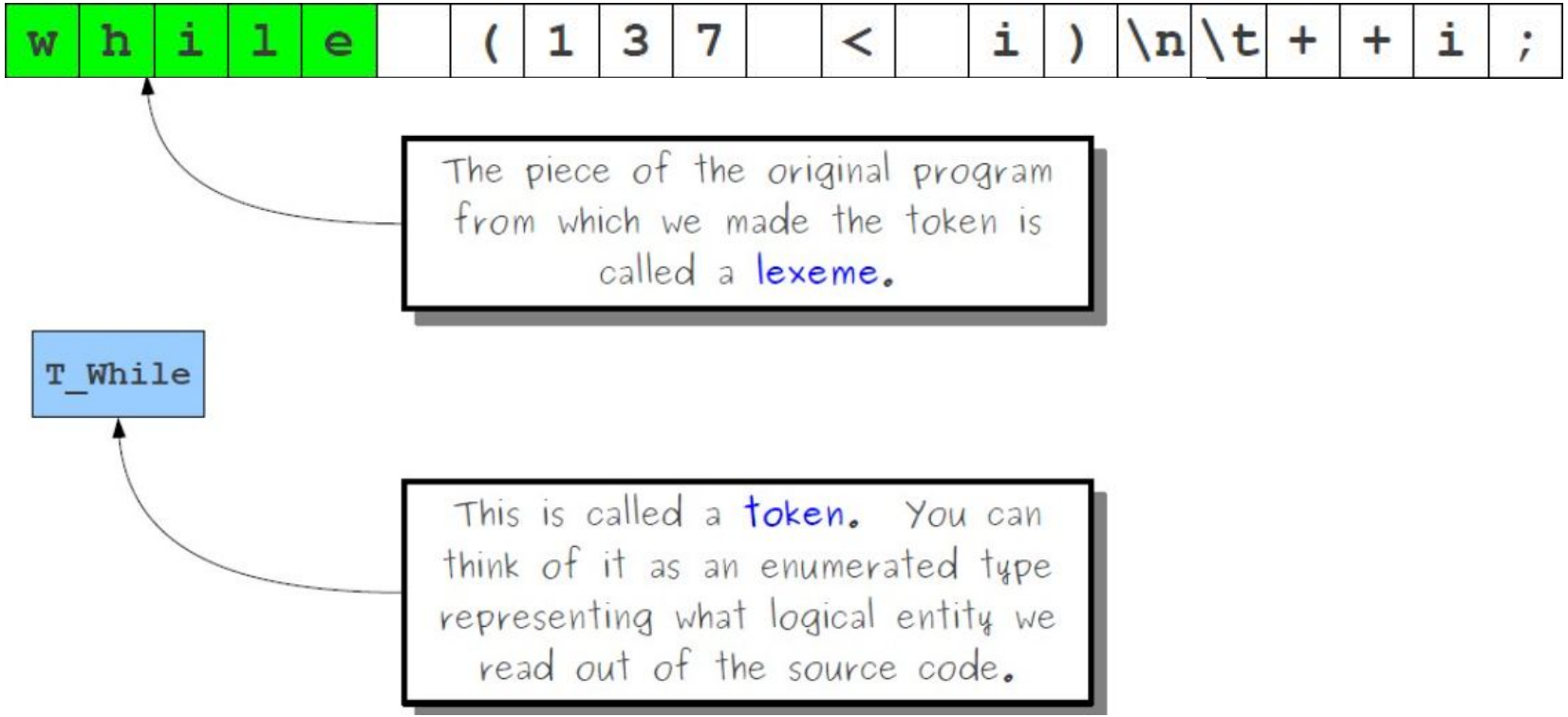
Scanning a Source File Example

w	h	i	l	e		(	1	3	7		<		i	)	\n	\t	+	+	i	;

Scanning a Source File Example



Scanning a Source File Example



Scanning a Source File Example



T_While

Scanning a Source File Example



T_While

Scanning a Source File Example



T_While

Scanning a Source File Example



T_While

Sometimes we will discard a lexeme rather than storing it for later use. Here, we ignore whitespace, since it has no bearing on the meaning of the program.

Scanning a Source File Example



T_While

Scanning a Source File Example



T_While

Scanning a Source File Example



T_While

Scanning a Source File Example





Scanning a Source File Example





Scanning a Source File Example



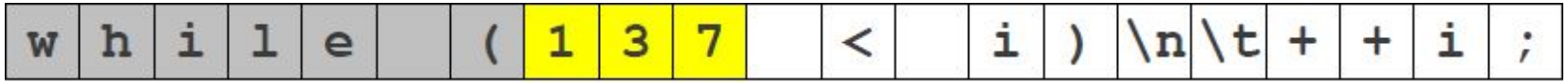


Scanning a Source File Example



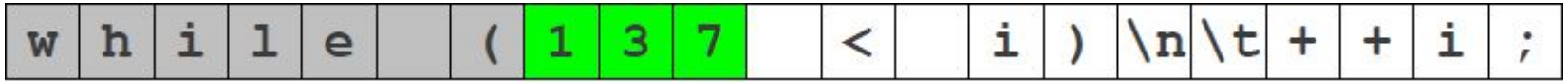


Scanning a Source File Example



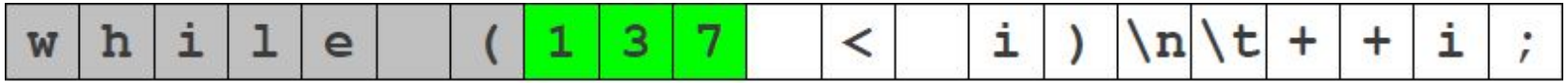


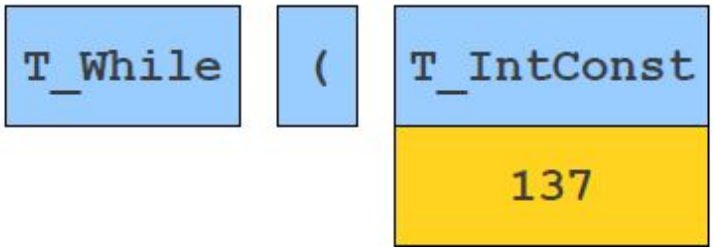
Scanning a Source File Example



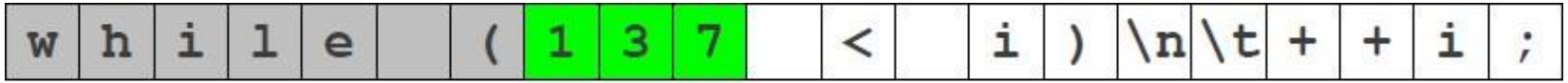


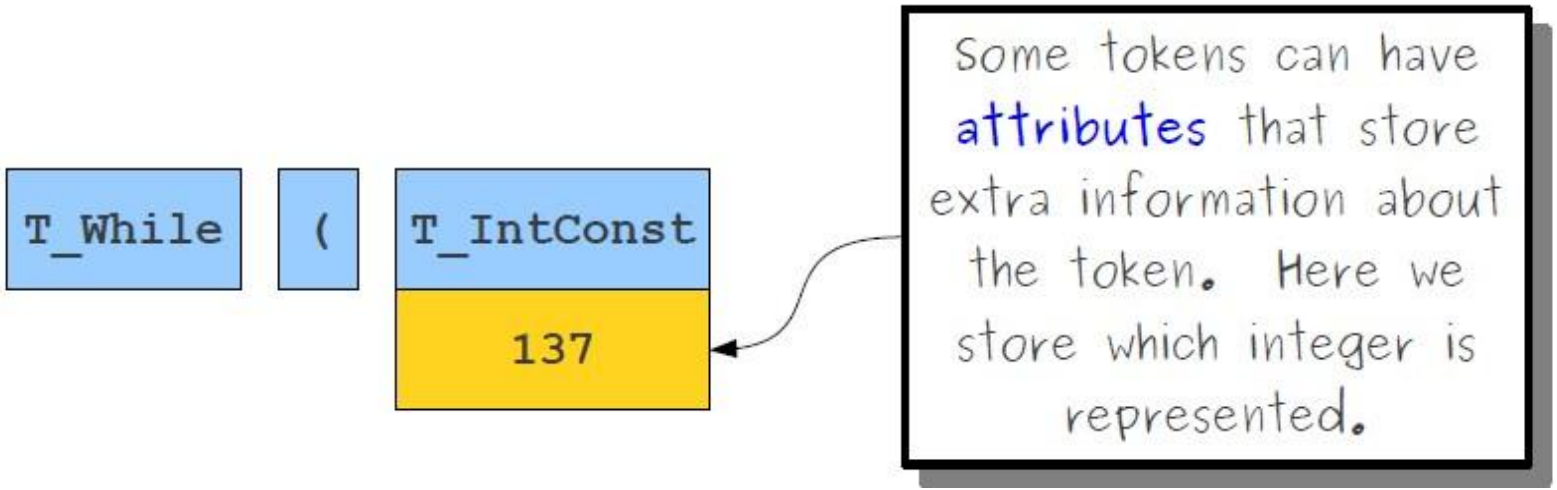
Scanning a Source File Example





Scanning a Source File Example





Scanning a Source File Example



.....And so on

Scanning a Source File Example 2



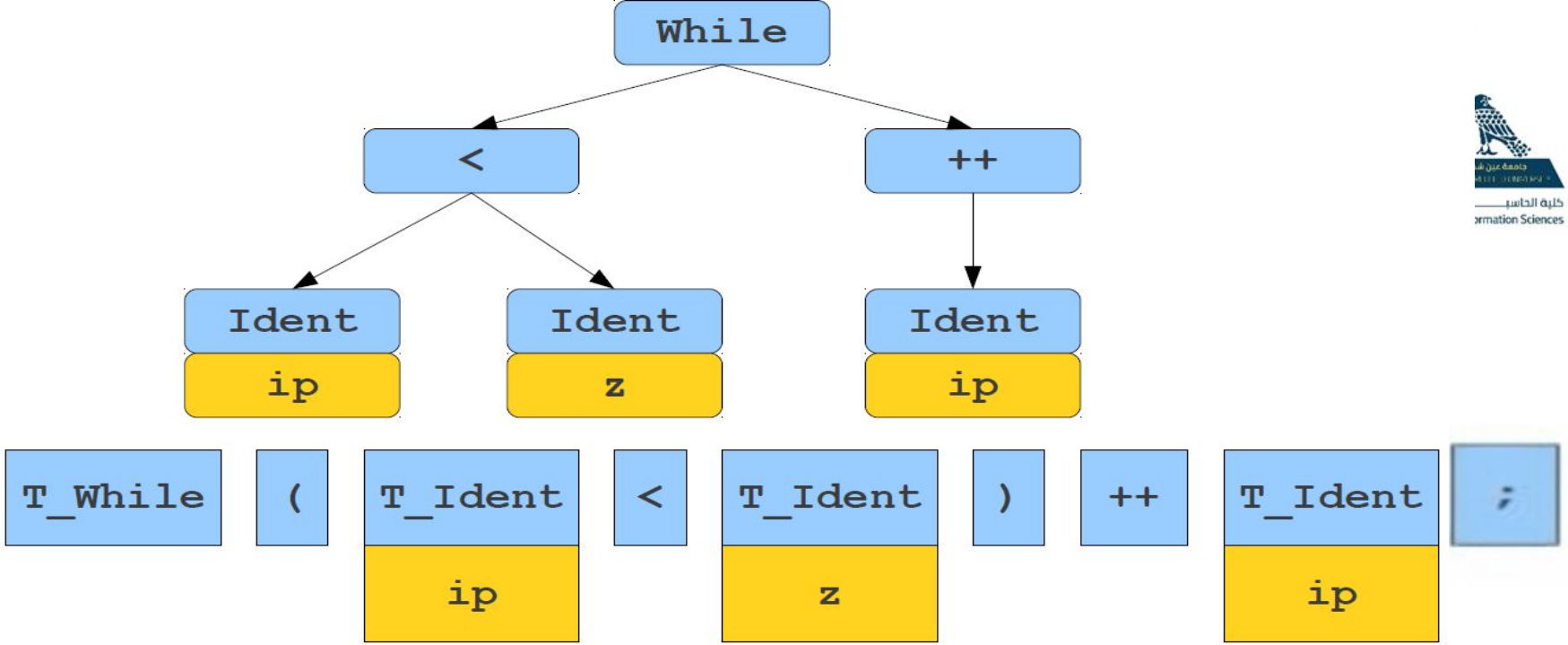

```
while (ip < z)
    ++ip;
```

Scanning a Source File Example 2



w	h	i	l	e		(	i	p		<		z	)	\n	\t	+	+	i	p	;

```
while (ip < z)
    ++ip;
```



w	h	i	l	e		(	i	p		<		z	)	\n	\t	+	+	i	p	;

```
while (ip < z)
++ip;
```

Token Definitions



- The Informal Approach:
  - "An identifier consists of a letter followed by any number of letters and digits".
- The Problem with Informality:
  - Informal descriptions are ambiguous and prone to edge-case bugs (e.g., Is _ a letter? Can a number have a trailing decimal point like 30.?).
- The Solution:
  - Regular Expressions: A rigorous mathematical language used to precisely define what is and is not permitted in each token category.



Regular Expressions

Regular Expressions (RE)



- A regular expression ***s*** is a string denoting a set of strings ***L(s)*** drawn from an alphabet ∑.
- The Language of ***s***: ***L(s)*** represents the "language of ***s***".
- Inductive Base Cases:
  - If ***a*** ∈ ∑, then ***a*** is a regular expression and ***L(a)*** = **{a}**.
  - **ϵ** (epsilon) is a regular expression representing the empty string: **L(ϵ) = {ϵ}**.

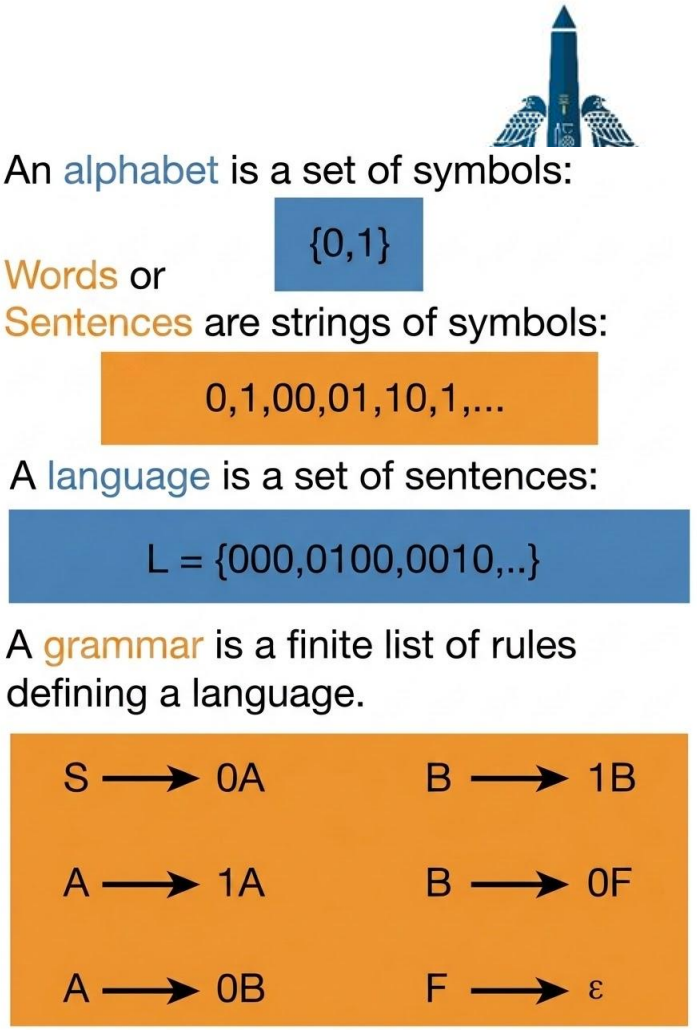
What is Automata Theory?



- Study of abstract computing devices, or “machines”
- Automaton = an abstract computing device.
  - Note: A “device” need not even be a physical hardware!

Languages & Grammars

- Languages: “A language is a collection of sentences of finite length all constructed from a finite alphabet of symbols”
- Grammars: “A grammar can be regarded as a device that enumerates the sentences of a language” - nothing more, nothing less.

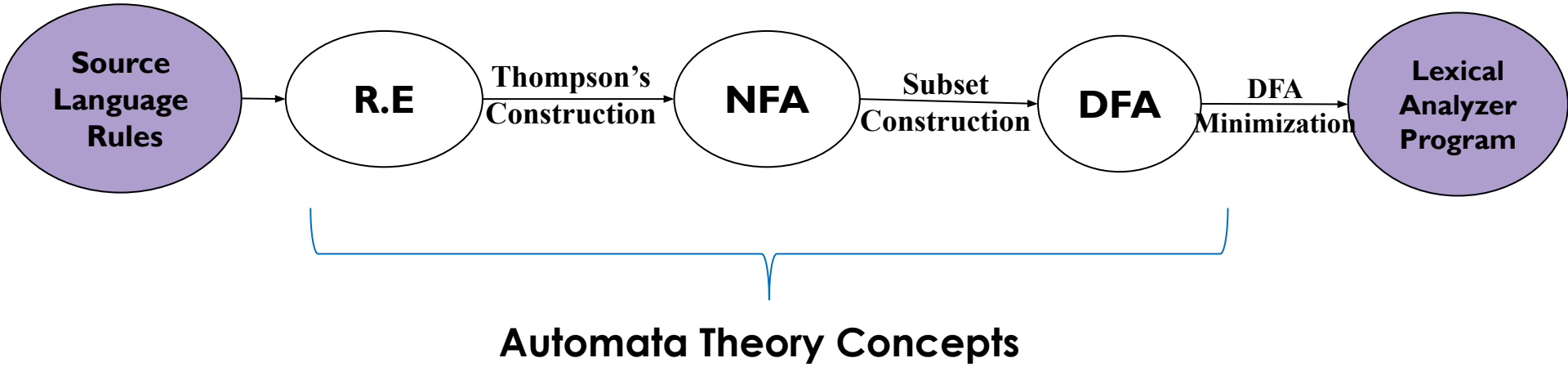


Formal Languages



- A **formal language** is a set of strings.
- Many infinite languages have **finite descriptions**:
  - Define the language using a **regular expression**.
  - Define the language using an **automaton**.
  - Define the language using a **grammar**.
- We can use these compact descriptions of the language to define sets of strings.
- Over the course of this class, we will use all of these approaches.

Lexical Analyzer and Automata



Alphabet

An alphabet is a finite, non-empty set of symbols

- We use the symbol ∑ (sigma) to denote an alphabet
- Examples:
  - Binary: ∑ = {0,1}
  - All lower case letters: ∑ = {a,b,c,..z}
  - Alphanumeric: ∑ = {a-z, A-Z, 0-9}
  - DNA molecule letters: ∑ = {a,c,g,t}
  - …

Strings

A string or word is a finite sequence of symbols chosen from ∑

- ***Empty string is ε (or "epsilon")***
- Length of a string *w*, denoted by "|*w*|", is equal to the *number of (non- ε) characters in the string*
  - *E.g., x = 010100* |*x*| = 6
  - *x = 01 ε 0 ε 1 ε 00 ε* |*x*| = 6

- *xy = concatenation* of two strings *x* and *y*

Alphabets and strings

- A common way to talk about words, number, pairs of words, etc. is by representing them as strings
- To define strings, we start with an alphabet
- Examples

$\Sigma_1 = \{a, b, c, d, \ldots, z\}$: the set of letters in English

$\Sigma_2 = \{0, 1, \ldots, 9\}$: the set of (base 10) digits

$\Sigma_3 = \{a, b, \ldots, z, \#\}$: the set of letters plus the special symbol #

$\Sigma_4 = \{ (,) \}$: the set of open and closed brackets

Strings

A **string** over alphabet Σ is a **finite sequence** of symbols from Σ.

- The **empty string** will be denoted by ε
- Examples

abfbz **is a string over** $\Sigma_1 = \{a, b, c, d, \ldots, z\}$

9021 **is a string over** $\Sigma_2 = \{0, 1, \ldots, 9\}$

ab#bc **is a string over** $\Sigma_3 = \{a, b, \ldots, z, \#\}$

))()(() **is a string over** $\Sigma_4 = \{ (,) \}$

Languages

- Languages can be used to describe problems with "yes/no" answers, for example:

A language is a set of strings over an alphabet.

L_1 = The set of all strings over Σ_1 that contain the substring "fool"

L_2 = The set of all strings over Σ_2 that are divisible by 7 $= \{7, 14, 21, \ldots\}$

L_3 = The set of all strings of the form s#s where s is any string over $\{a, b, \ldots, z\}$

L_4 = The set of all strings over Σ_4 where every (can be matched with a subsequent)

Basic Concepts

- **Regular expressions**
  - represent patterns of strings of characters.
- **A regular expression *r***
  - **completely** defines by the set of strings it matches.
  - The set is called the language of **r** written as **L(r)**
- **The set of elements**
  - referred to as **symbols**
- **The set of legal symbols**
  - called the **alphabet** and written as the Greek symbol sigma ∑

Basic Concepts



- A regular expression ***r***
  - contains characters from the alphabet, indicating patterns, such as ***a;*** which is the character **a** used as a pattern.
  - may contain **special characters** called *meta-characters* or meta-symbols (e.g. **|** *****)
- An *escape character* can be used to turn off the special meaning of a meta-character.
- Such as backslash and quotes.
- E.g. * matches *

Basic Concepts

- The single character from alphabet matches itself in the Language, as follows:
  - **a** matches the character ***α*** by writing **L(a)={ a }**
  - **ε** denotes the empty string, by **L(ε)={ε}**
  - **{}** or **Φ** matches no string at all, by **L(Φ)={ }**

Basic Concepts



Operations in Regular Expressions:

- **Choice:** among alternatives, indicated by the meta-character |
- **Concatenation:** indicated by juxtaposition
- **Repetition** or "**closure**": indicated by the meta-character *

Basic Concepts



Choice Among Alternatives:

- If **r** and **s** are regular expressions, then **r|s** is a regular expression which matches any string that is matched either by **r** or by **s**.
- In terms of languages, the language **r|s** is the union of language **r** and **s**, or **L(r|s)=L(r) U L(s)**
- A simple example: **L(a|b)=L(a)U L(b)={a, b}**
- Choice can be extended to more than one alternative.

Basic Concepts



Concatenation:

- If **r** and **s** are **regular expressions**, then **rs** is their concatenation which matches any string that is the concatenation of two strings, the first of which matches **r** and the second of which matches **s**.
- In terms of generated **languages**, the concatenation set of strings **S1S2** is the set of strings of S1 appended by all the strings of S2.
- A simple example**: L(a|b)c ={ac, bc}**
- Concatenation can also be extended to more than two regular expressions.

Basic Concepts

جامعة عين شمس
AIN SHAMS UNIVERSITY
كلية الحاسبات والمعلومات
Faculty of Computer & Information Sciences

Repetition:

- The repetition operation of a regular expression, called (**Kleene**) **closure**, is written **r*,** where **r** is a regular expression.
  - The regular expression **r*** matches any finite concatenation of strings, each of which matches r.
- A simple example, **a*** matches the strings

  ε, a, aa, aaa,…

- In term of generated language, given a set of **S** of string, **S*** is an infinite set union, but each element in it is a finite concatenation of string from S.

Basic Concepts



Precedence of Operations and Use of Parentheses:

From Higher to Lower:

1. **Repetition** *
2. **Concatenation**
3. **Choice** | is given the lowest

- A simple example

  a|bc* is interpreted as **a|(b(c*))**

- **Note:** *Parentheses is used to indicate a different precedence.*

Basic Concepts

جامعة عين شمس
AIN SHAMS UNIVERSITY
كلية الحاسبات والمعلومات
Faculty of Computer & Information Sciences

Name for regular expression:

- **You can give a name to a long regular expressions**
  - ***digit* = 0|1|2|3|4.....|9**
  - **(0|1|2|3.....|9)(0|1|2|3.....|9)***
  - ***digit digit****

Definition of Regular Expression

جامعة عين شمس
AIN SHAMS UNIVERSITY
كلية الحاسبات والمعلومات
Faculty of Computer & Information Sciences

To sum up, a regular expression is one of the following:

(1) A **basic regular expression**: a single legal character **a** from alphabet **∑**, or meta-character **ε** or **Φ.**

(2) The form **r|s**, where r and s are regular expressions.

(3) The form **rs**, where r and s are regular expressions.

(4) The form **r***, where r is a regular expression.

(5) The form **(r)**, where r is a regular expression.

Examples of Regular Expressions

جامعة عين شمس
AIN SHAMS UNIVERSITY
كلية الحاسبات والمعلومات
Faculty of Computer & Information Sciences

Example 1:

- ∑={ a,b,c}
- Write a regular expression describing the set of all strings over this alphabet that contains exactly one b.

(a|c)*b(a|c)*

Examples of Regular Expressions

جامعة عين شمس
AIN SHAMS UNIVERSITY
كلية الحاسبات والمعلومات
Faculty of Computer & Information Sciences

Example 2:

- ∑={ a,b,c}
- The set of all strings that contains at most one b.
  - (a|c)*|((a|c)*b(a|c)*)
  - (a|c)*(b|ε)(a|c)*

The same language may be generated by many different regular expressions.

Examples of Regular Expressions

Example 3:

- ∑={ a,b}
- The set of strings consisting of a single b surrounded by the same number of a's.

  S = {b, aba, aabaa,aaabaaa,……} = { $a^n b a^n$ | n≠0}

This set can NOT be described by a regular expression.

"regular expression can NOT count !!!"

not all sets of strings can be generated by regular expressions.

A regular set :

✔ a set of strings which is the language for a regular expression.

Examples of Regular Expressions

Example 4:

- ∑={ a,b,c}
- **The strings that contain no two consecutive b's**

((a|c)* | (b(a|c))*)*

((a | c) | (b(a | c)))* or (a | c | ba | bc)*



The correct regular expression

✔ **(a | c | ba | bc)* (b |ε)**

✔ **((b |ε) (a | c | ab| cb)***

✔ **(not b |b not b)*(b|ε)**

Examples of Regular Expressions

Example 5:

- ∑={ a,b,c}

((b|c)* a(b|c)*a)* (b|c)*

- **Determine a concise English description for the language of the given regular expression**

✔ **Strings containing an even number of a's**

(nota* a nota* a)* nota*

List of New Operations

1. one or more repetitions

   r^+

2. a range of characters

   [0-9], [a-zA-Z]

3. any character

   period .

4. any character not in a given set

   ~(a|b|c) a character not either a or b or c

5. optional sub-expressions

   r? the strings matched by r are optional



Regular Expressions for Programming Language Tokens

Number, Reserved word and Identifiers

- **Numbers**
  - *naturalNumber*= [**0-9**]$^{+}$
  - *signedNaturalNumber* = (**+**|**-**)?*naturalNumber*
  - *number* = *signedNaturalNumber*(**\.***naturalNumber*)?(**E** *signedNaturalNumber*)?
- **Reserved Words and Identifiers**
  - *reserved* = **if** | **while** | **do** |.....
  - *letter* = [**a-z** |**A-Z**]
  - *digit* = [**0-9**]
  - *identifier* = *(letter*|_ *)(letter*|*digit*|_ *)**

White Space



- **White space:**
  - Delimiters = characters that are unambiguously part of other tokens are delimiters.
  - whitespace = (***newline*** | ***blank*** | ***tab*** | ***comment***)$^{+}$

Ambiguity

- Some strings can be matched by several different regular expressions.
  - either an **identifier** or a **keyword**, **keyword interpretation preferred**.

    e.g. **for** is it a keyword or a variable?
  - a single token or a sequence of several tokens, the single-token preferred.

    e.g. **ifg=3;** is it a variable named “ifg”? Or should it interpret it as starting with the keyword “if” followed by the variable “g”?? (The principle of **longest sub-string.**)

Implementing Regular Expressions



- Regular expressions can be implemented using finite automata.
- There are two main kinds of finite automata:

a) NFAs (nondeterministic finite automata),

b) DFAs (deterministic finite automata)

Lexical Analyzer and Automata

